

Data Engineering II (Charges data-intensives) — Labs & Projet

Author : Badr TAJINI

Année académique : 2025–2026

Ecole : ESIEE Paris

Cours : Data Engineering II (Charges data-intensives)

1. Objectifs du cours

- Concevoir et instrumenter des **pipelines streaming** (Structured Streaming, fenêtres, watermarks, sinks Parquet) et mesurer la latence et le débit.
- Construire des **pipelines de traitement de texte** (tokenisation, index inversé, TF-IDF) et mesurer les coûts d'I/O, la latence de requête et l'empreinte de stockage.
- Implémenter et évaluer des **charges itératives** : soit en traitement de graphes (PageRank, connected components), soit en clustering (KMeans, BisectingKMeans) : avec une attention particulière au partitionnement, aux coûts de shuffle et à la convergence.
- Préparer une **infrastructure de données pour les LLMs** du point de vue plateforme : curation, filtres qualité, design de schéma, versionnage.
- Appliquer une **optimisation fondée sur des preuves** : `explain("formatted")`, Spark UI, journaux de métriques, comparaisons avant / après.

2. Compétences acquises

- Associer des **charges data-intensives** (streaming, texte, graphes, clustering) à des représentations et layouts adaptés.
- Instrumenter chaque étape avec `explain("formatted")`, Spark UI et journaux de métriques.
- Mesurer et comparer les **coûts** (shuffle, I/O, spill) avant et après optimisation, avec des preuves quantitatives.
- Proposer des **améliorations défendables** alignées avec des SLO mesurables.

3. Prérequis

- Validation de **Data Engineering I** (ETL, plans Spark, Parquet, métriques).
- Python et SQL de base.
- Familiarité avec le terminal, Git et conda.

4. Contenu du cours (6 chapitres)

1. **Text Processing I** : ingestion de texte, normalisation, tokenisation, index inversé, représentations vectorielles creuses, pondération.
2. **Text Processing II** : représentations denses, top-k retrieval, rerankers, RAG, MCP, relation sparse / dense.

3. **Finding Similar Items** : collisions de hachage, MinHash, LSH, random projections, clustering (KMeans, GMM), banding.
4. **Graph Processing** : représentations de graphes, algorithmes (PageRank, BFS), edge partitioning, coûts de communication, skew.
5. **Stream Processing** : micro-batch vs continu, fenêtres, watermarks, sémantiques de livraison, tolérance aux pannes, intégration lakehouse.
6. **LLMs & infrastructure data** : curation des données, tables de features, fraîcheur, versionnage, gouvernance ; angle plateforme, pas angle algorithmes.

5. Techniques et méthodes

- **Mesure systématique** : `explain("formatted")`, Spark UI (Files Read, Input Size, Shuffle Read/Write, Spill).
- **Comparaisons avant / après** : même seed, même échantillon, mêmes conditions.
- **Contrats de schéma** à l'ingestion, conversion colonnaire, partitionnement guidé par l'usage.
- **Reproductibilité locale** : conda, Java 21, Spark 4, notebooks scriptables, seeds fixes.
- **Journal d'expérience** : `CHANGELOG.md`, `EXPERIMENTS.md`, `PLAN_EVIDENCE.md`.

6. Outils techniques

- **Installation locale** : conda (Python 3.10), OpenJDK 21, Maven, JupyterLab.
- **Spark** : Spark 4.0.0 précompilé ; PySpark dans Jupyter et via `spark-submit`.
- **MLlib** : KMeans, BisectingKMeans, ClusteringEvaluator, VectorAssembler (clustering uniquement ; pas de ML supervisé).
- **Structured Streaming** : micro-batch, sinks fichier / console, watermarks, trigger intervals.
- **GraphFrames** (optionnel) : pour les algorithmes de graphe ; ou joins itératifs manuels.
- **Vérification** : session Spark, lecture CSV / Parquet, comparaison de plans, captures Spark UI.

7. Acquis d'apprentissage

À l'issue du cours, l'étudiant est capable de :

- Construire des **pipelines data-intensifs** (streaming, texte, graphes / clustering) et produire des **plans** et **métriques** comme preuves.
- Expliquer les **trade-offs d'architecture** pour chaque type de charge et défendre ses **choix** par des mesures.
- Documenter la **reproductibilité**, la **traçabilité** et la **qualité** de ses pipelines.

8. Évaluation

- **Labs** : 40 %
 - **Lab 1** (10 %) : Streaming : pipeline Structured Streaming, fenêtres, watermarks, sink Parquet, monitoring, optimisation avant / après
 - **Lab 2** (15 %) : Traitement de texte : ingestion de corpus, normalisation, index inversé, latence de requête, empreinte disque (CSV vs Parquet)
 - **Lab 3** (15 %) : Charge itérative (Graphes OU Clustering) : stratégies de partitionnement, métriques par itération, convergence, analyse du skew, rapport avant / après
- **Projet final** : 30 %

- **Partie A** (15 %) : pipeline end-to-end (batch ETL + streaming), SLO, contrats de schéma, plans et métriques de base
- **Partie B** (15 %) : pipeline texte, charge itérative, préparation LLM, optimisation de layout, rapport + preuves
- **Documentation** (rapports et preuves) : 20 %
- **Participation et Q&A** : 10 %

Lab 0 — Bootstrap (non noté)

Revalider l'environnement local, tester PySpark, lire le plan du cours et collecter les premières métriques. Sert de point de départ pour les labs suivants.

Labs notés (trois niveaux progressifs)

- **Lab 1** : pipeline Structured Streaming avec source fichier / socket, agrégation fenêtrée, watermarks, sink Parquet, `query.lastProgress`, Streaming UI, optimisation avant / après.
- **Lab 2** : traitement de texte : ingestion de corpus, tokenisation, normalisation, suppression de stop words, construction d'un index inversé, mesure de latence, comparaison d'empreinte Parquet vs CSV.
- **Lab 3** : charge itérative : au choix **graph processing** (PageRank via joins itératifs ou GraphFrames) OU **clustering** (sweep KMeans / BisectingKMeans). Dans les deux cas : partitionnement, métriques par itération, convergence, rapport avant / après.

Projet final (deux parties)

- **Partie A** : pipeline end-to-end, batch ETL (bronze → silver → gold), ingestion streaming, SLO, calibrations de base, preuves par plans et métriques.
- **Partie B** : pipeline texte, charge itérative (graphes ou clustering), préparation de données pour LLMs, optimisation du layout (compaction, partitionnement), rapport ≤ 8 pages.

4 pistes thématiques

Au choix **une piste** et vous la conservez pour les Labs 1–3 et le projet :

1. **Track A — Esport (OpenDota)** : matchs, héros, archétypes, impact des patches
2. **Track B — Open Source (GitHub Archive)** : événements, PR, rythmes de release
3. **Track C — Micromobilité (Citi Bike)** : trajets, stations, stock-out, résilience
4. **Track D — Aviation (OpenSky)** : segments de vol, aéroports, régimes d'espace aérien

9. Références

- **Ouvrages** : *Designing Data-Intensive Applications (2nd ed.)* ; *Fundamentals of Data Engineering* ; *Mining of Massive Datasets* (chap. 3, 5).
- **Technique** : documentation PySpark, Spark MLlib, guide Structured Streaming.
- **Recherche d'information texte** : *Data-Intensive Text Processing with MapReduce* (chap. 4–5) ; *Pretrained Transformers for Text Ranking*.
- **Streaming** : Apache Beam : Streaming 101 & 102 ; *Kafka Streams in Action* (chap. 1–2).
- **Graphes** : *Scale Up or Scale Out for Graph Processing?* ; article COST (McSherry et al.).
- **Datasets** : OpenDota, GitHub Archive, Citi Bike, OpenSky, Wikipedia Clickstream, Open Food Facts.

Livrables

- Notebooks propres et exécutables
- **Captures Spark UI**
- Tableaux de métriques **avant / après**
- **Rapports concis et traçables** (hypothèses, choix, preuves, limites, recommandations)
- **GENAI .md** déclarant l'usage éventuel d'IA générative